

Machine execution

Jinyang Li

Lesson Plan: last time

- Basic h/w execution model:
 - CPU fetch the next instruction from memory according to PC
 - Decode and execute instruction (e.g. mov instruction)
 - CPU updates PC to point to next instruction
- ISA (instruction set architecture): x86, ARM, RISC-V
- X86 ISA
 - %rip (PC), 16 general purpose registers
 - mov instruction

Lesson Plan: today

- mov
 - more memory addressing modes
- lea
- arithmetic instructions
- How CPUs realize non-linear control flow

x86 instruction: Moving (aka copying) data

movq *Source, Dest*

- Copy a quadword (64-bit) from the source operand (first operand) to the destination operand (second operand).

We use AT&T (instead of Intel) syntax for assembly

mov instruction

mov^{suffix}**q** *Source, Dest*

- Copy a quadword (8-bytes) from the source operand to the destination operand.

Suffix	Name	Size (byte)
b	Byte	1
w	Word	2
l	Long	4
q	Quadword	8

Why using a size suffix?

movq *Source, Dest*

- Support **full backward compatibility**
 - New processor can run the same binary file compiled for older processors
- In the Intel x86 world, a word = 16 bits.
 - 8086 refers to 16 bits as a word

mov instruction

movq *Source, Dest*

Operand Types

- **Immediate:** Constant integer data
 - Prefixed with \$
 - E.g: \$0x400, \$-533
- **Register:** One of general purpose registers
 - E.g: %rax, %rsi
- **Memory:** 8 consecutive bytes of memory
 - Indexed by register with various “address modes”
 - Simplest example: (%rax)

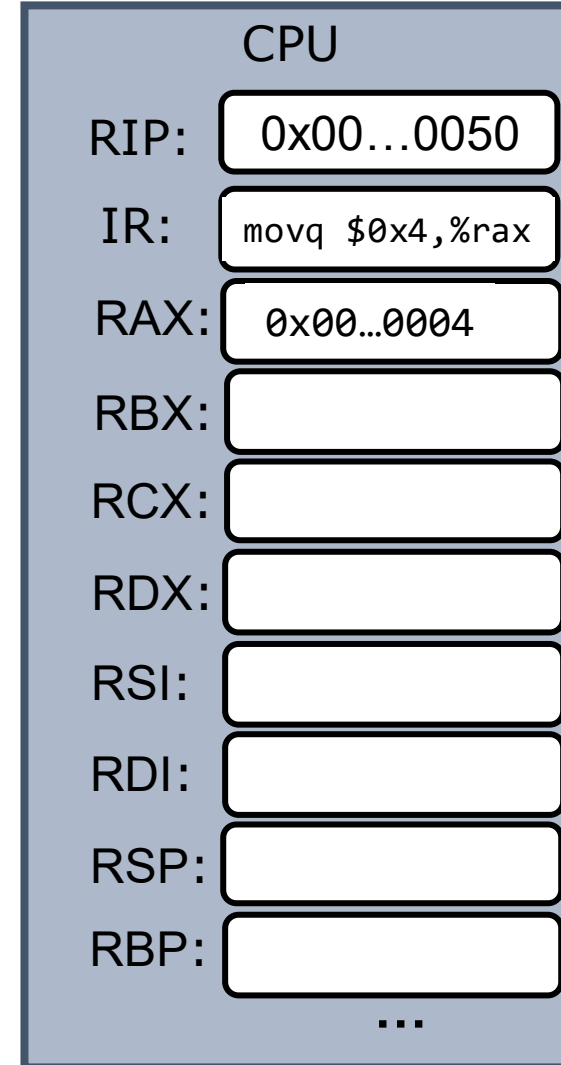
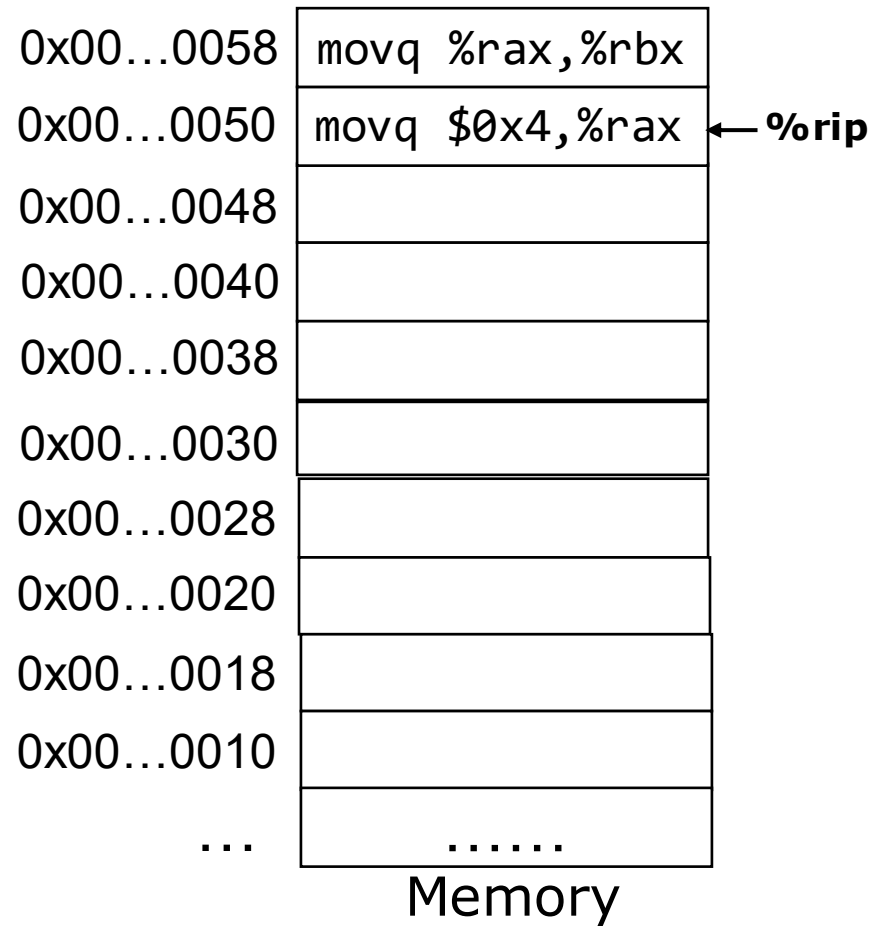
mov Operand combinations

	Source	Dest	Example
movq	Imm	Reg	movq \$0x4,%rax
		Mem	movq \$0x4, (%rax)
	Reg	Reg	movq %rax,%rdx
		Mem	movq %rax, (%rdx)
	Mem	Reg	
			movq (%rax),%rdx

1. Immediate can only be *source*

2. No memory-memory mov

movq *Imm, Reg*



movq *Reg, Reg*

0x00...0058	movq %rax,%rbx	← %rip
0x00...0050	movq \$0x4,%rax	
0x00...0048		
0x00...0040		
0x00...0038		
0x00...0030		
0x00...0028		
0x00...0020		
0x00...0018		
0x00...0010		
...		

Memory

CPU	
RIP:	0x00...0058
IR:	movq %rax, %rbx
RAX:	0x00...0004
RBX:	0x00...0004
RCX:	
RDX:	
RSI:	
RDI:	
RSP:	
RBP:	
...	

`movq Mem, Reg`

How to represent a “memory” operand?

Direct addressing: use register to index memory

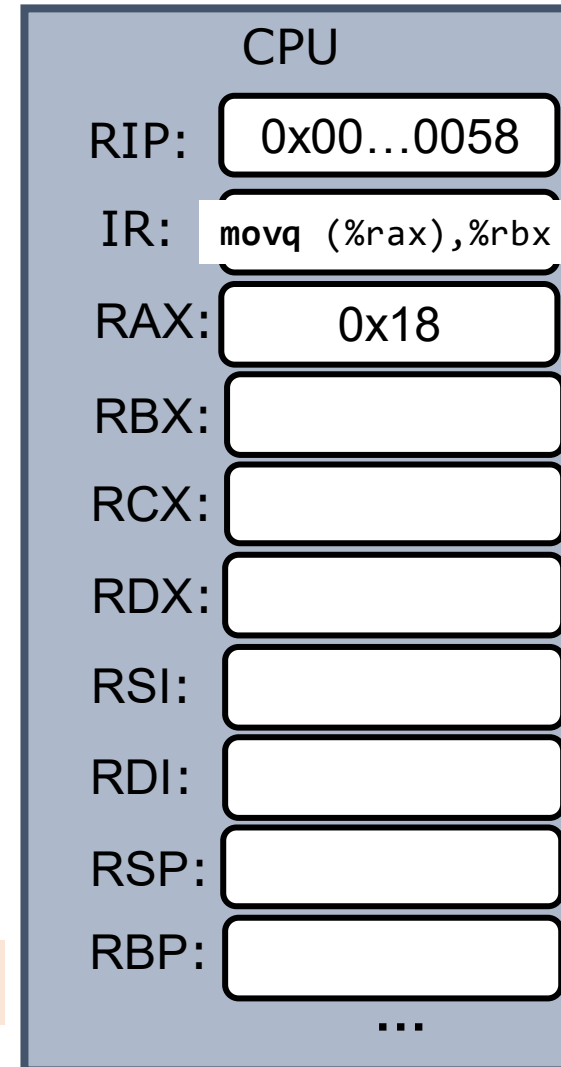
(Register)

- The content of the register specifies memory address
- `movq (%rax), %rbx`

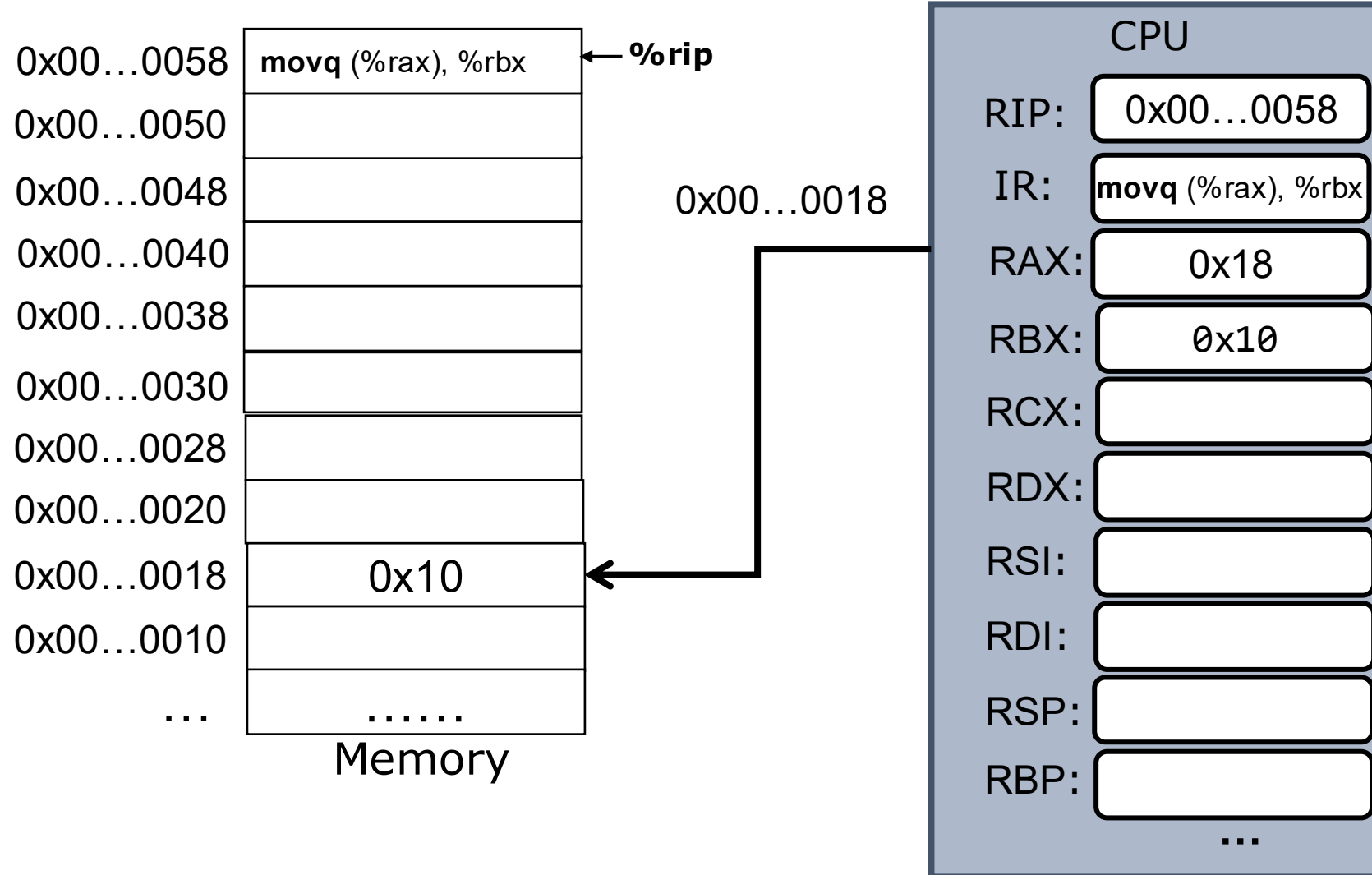
movq (%rax), %rbx

0x00...0058	movq (%rax), %rbx	← %rip
0x00...0050		
0x00...0048		
0x00...0040		
0x00...0038		
0x00...0030		
0x00...0028		
0x00...0020		
0x00...0018	0x10	
0x00...0010		
...		

How many bytes are copied? Source? Destination?



movq (%rax), %rbx



swap function

```
void  
swap(long *a, long* b) {  
  
    long tmp = *a;  
    *a = *b;  
    *b = tmp;  
  
}
```

swap:

gcc   -O3 swap.c

Makes gcc output assembly
(human readable machine instructions)

swap function

```
void  
swap(long *a, long* b) {  
    long tmp = *a;  
    *a = *b;  
    *b = tmp;  
}
```



gcc -S -O3 swap.c

swap:

```
movq    (%rdi), %rax  
movq    (%rsi), %rdx  
movq    %rdx, (%rdi)  
movq    %rax, (%rsi)
```

%rdi stores a

%rsi stores b

%rax is local variable tmp

swap function

```
void  
swap(long *a, long* b) {  
  
    long tmp = *a;  
    *a = *b;  
    *b = tmp;  
  
}
```



gcc -S -O3 swap.c

swap:

```
movq    (%rdi), %rax  
movq    (%rsi), %rdx  
movq    %rdx, (%rdi)  
movq    %rax, (%rsi)
```

Use two instructions and %rdx to perform
memory to memory move

swap function

```
void  
swap(long *a, long* b) {  
  
    long tmp = *a;  
    *a = *b;  
    *b = tmp;  
  
}
```



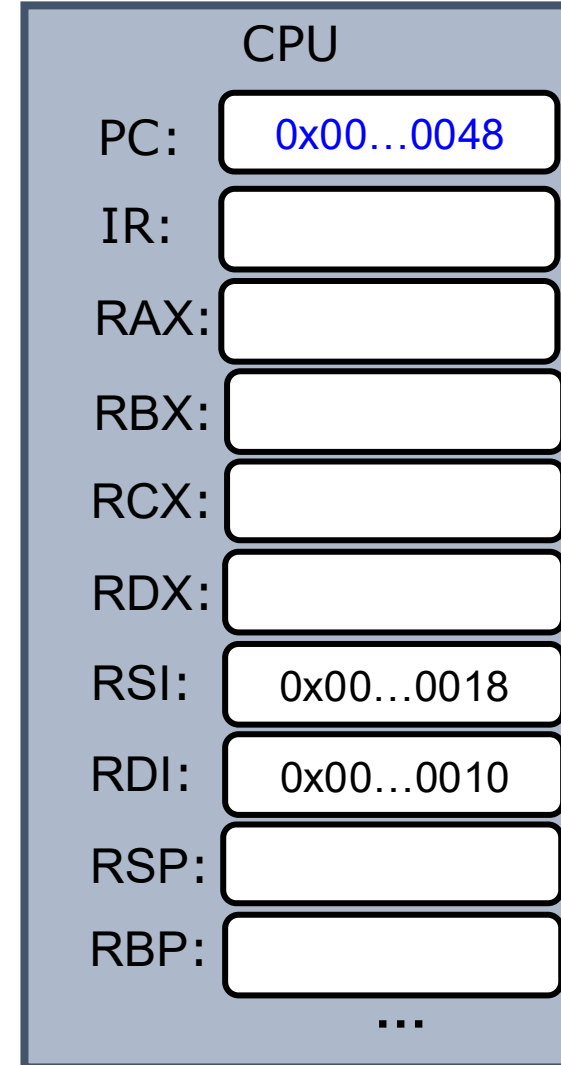
gcc -S -O3 swap.c

```
swap:  
    movq    (%rdi), %rax  
    movq    (%rsi), %rdx  
    movq    %rdx, (%rdi)  
    movq    %rax, (%rsi)
```

swap func

	0x00...0060	movq %rax, (%rsi)	
	0x00...0058	movq %rdx, (%rdi)	
	0x00...0050	movq (%rsi), %rdx	
	0x00...0048	movq (%rdi), %rax	← PC
	0x00...0040		
	0x00...0038		
	0x00...0030		
	0x00...0028		
	0x00...0020		
main.y:	0x00...0018	0x2	
main.x:	0x00...0010	0x1	
	...		

Memory



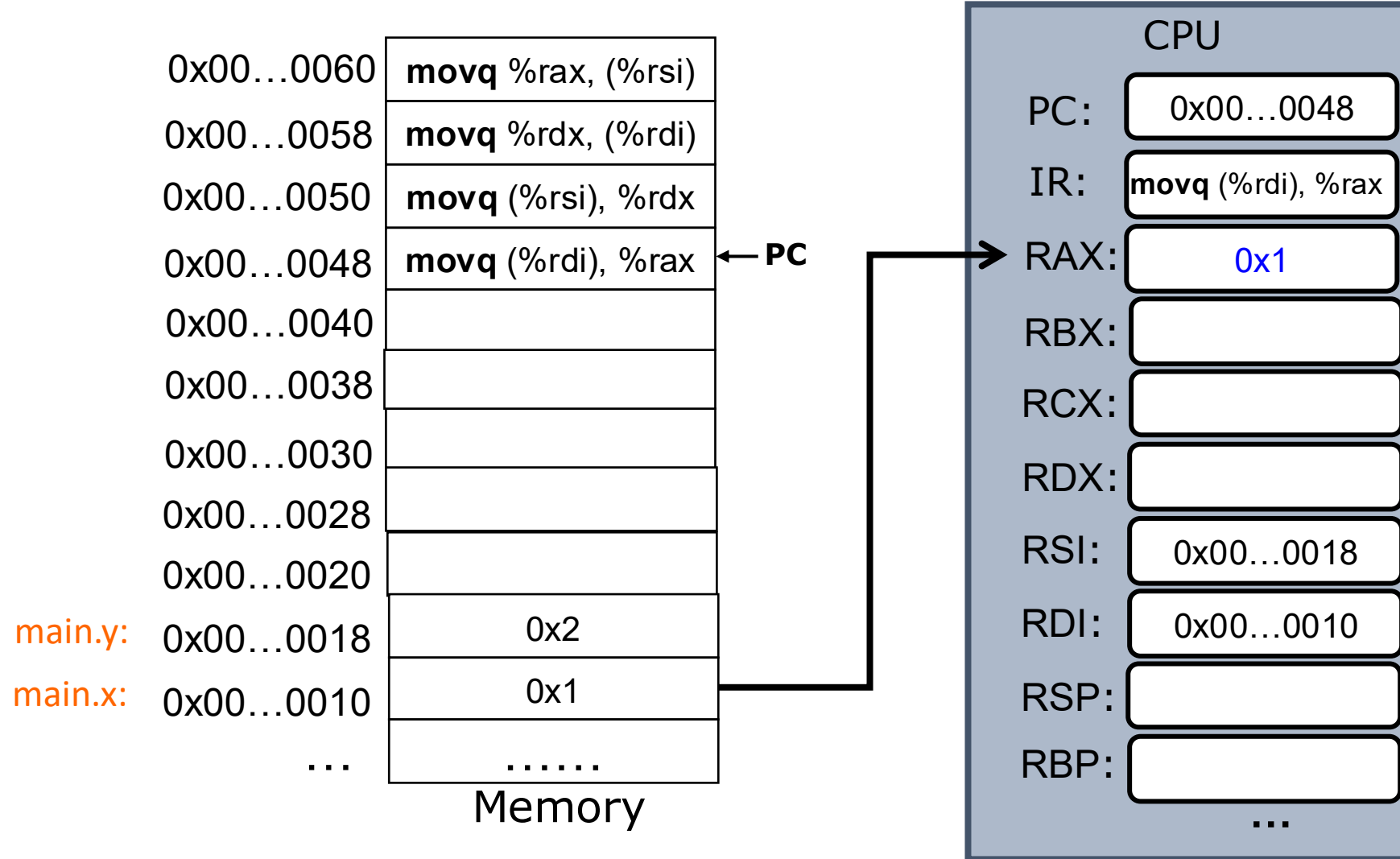
swap func

	0x00...0060	movq %rax, (%rsi)	
	0x00...0058	movq %rdx, (%rdi)	
	0x00...0050	movq (%rsi), %rdx	
	0x00...0048	movq (%rdi), %rax	← PC
	0x00...0040		
	0x00...0038		
	0x00...0030		
	0x00...0028		
	0x00...0020		
main.y:	0x00...0018	0x2	
main.x:	0x00...0010	0x1	
	...		

Memory

CPU	
PC:	0x00...0048
IR:	movq (%rdi), %rax
RAX:	
RBX:	
RCX:	
RDX:	
RSI:	0x00...0018
RDI:	0x00...0010
RSP:	
RBP:	
	...

swap func



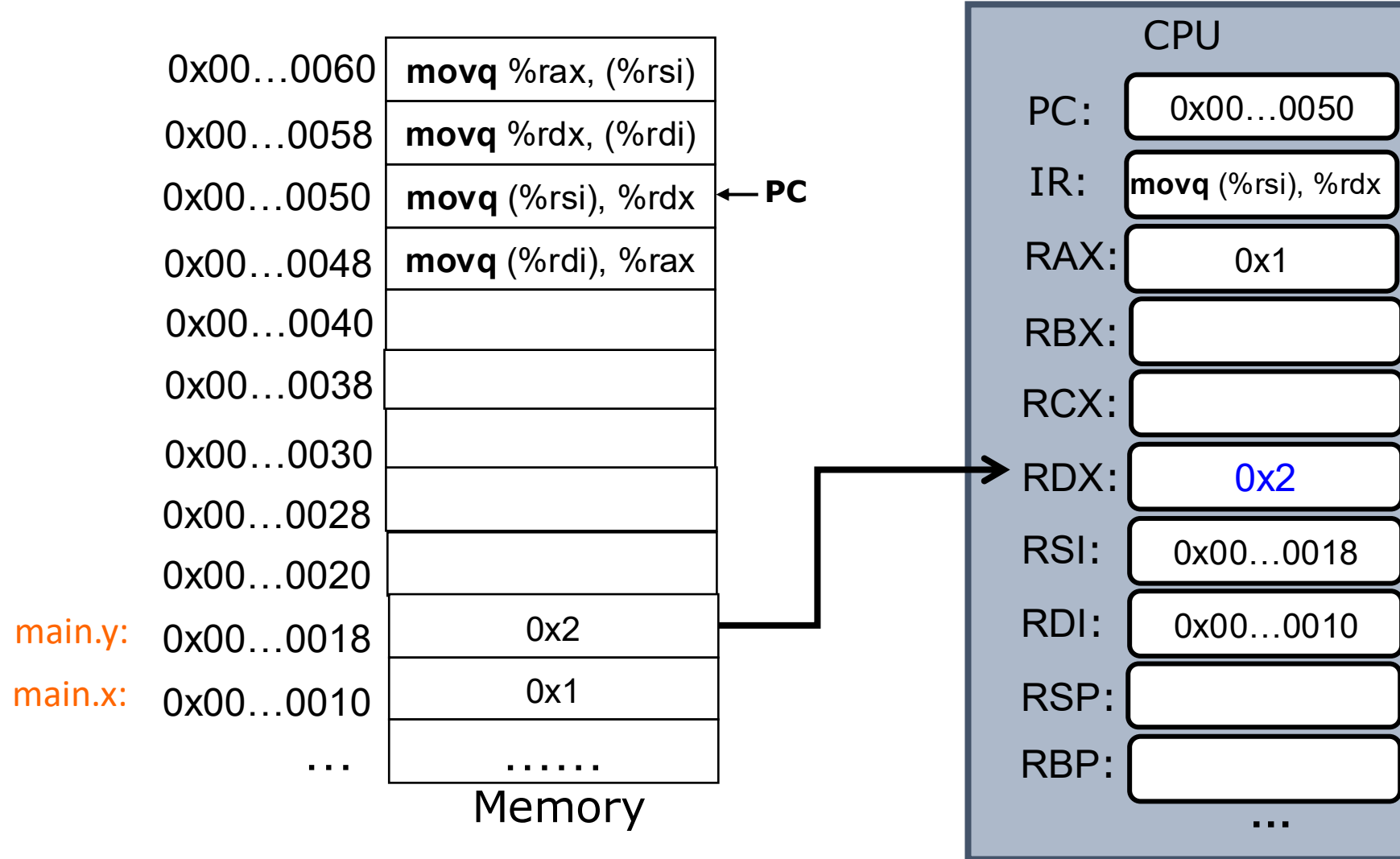
swap func

	0x00...0060	movq %rax, (%rsi)	
	0x00...0058	movq %rdx, (%rdi)	
	0x00...0050	movq (%rsi), %rdx	← PC
	0x00...0048	movq (%rdi), %rax	
	0x00...0040		
	0x00...0038		
	0x00...0030		
	0x00...0028		
	0x00...0020		
main.y:	0x00...0018	0x2	
main.x:	0x00...0010	0x1	
	...		

Memory

CPU	
PC:	0x00...0050
IR:	movq (%rsi), %rdx
RAX:	0x1
RBX:	
RCX:	
RDX:	
RSI:	0x00...0018
RDI:	0x00...0010
RSP:	
RBP:	
	...

swap func



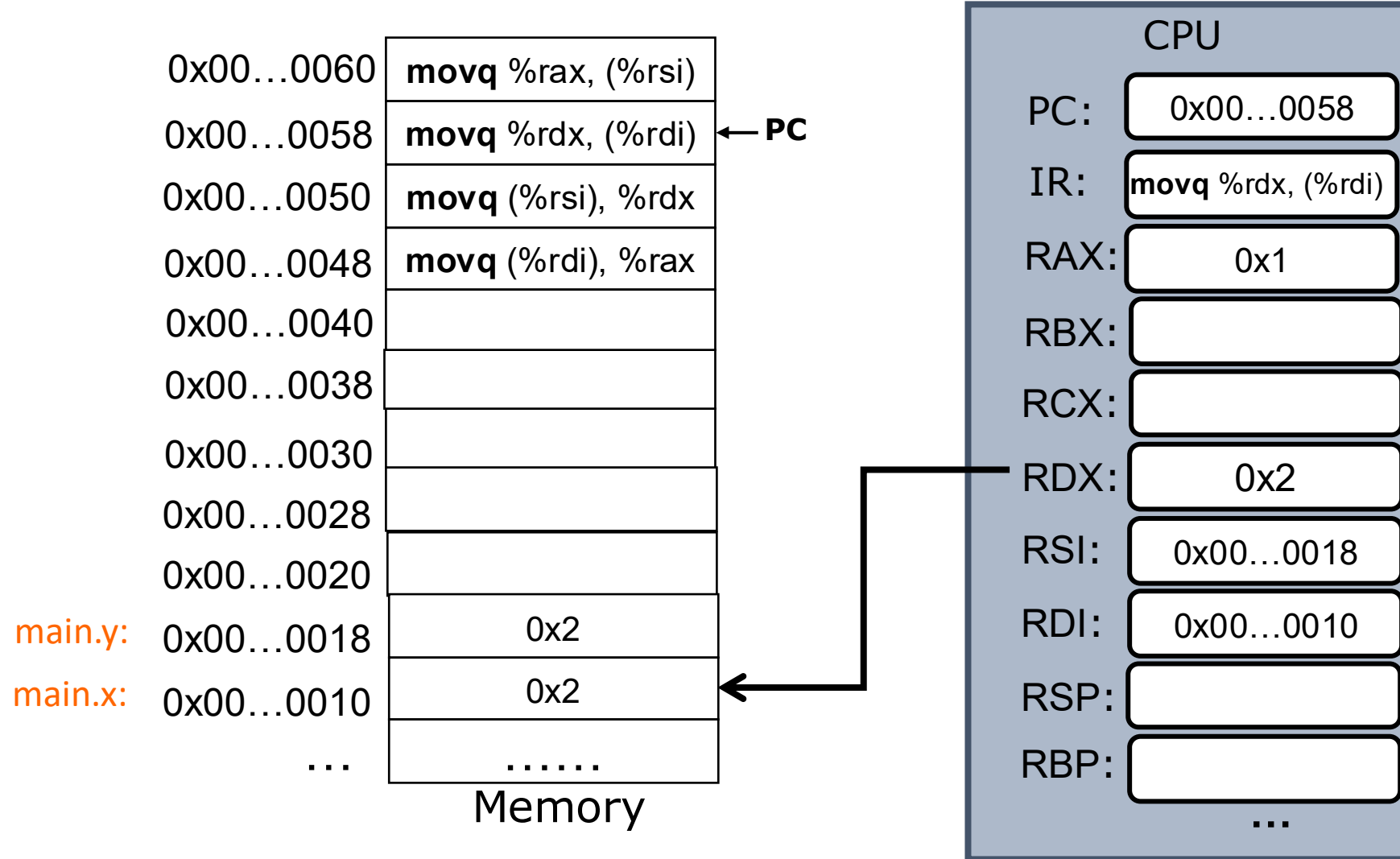
swap func

	0x00...0060	movq %rax, (%rsi)	
	0x00...0058	movq %rdx, (%rdi) ← PC	
	0x00...0050	movq (%rsi), %rdx	
	0x00...0048	movq (%rdi), %rax	
	0x00...0040		
	0x00...0038		
	0x00...0030		
	0x00...0028		
	0x00...0020		
main.y:	0x00...0018	0x2	
main.x:	0x00...0010	0x1	
	...		

Memory

CPU	
PC:	0x00...0058
IR:	movq %rdx, (%rdi)
RAX:	0x1
RBX:	
RCX:	
RDX:	0x2
RSI:	0x00...0018
RDI:	0x00...0010
RSP:	
RBP:	
...	

swap func



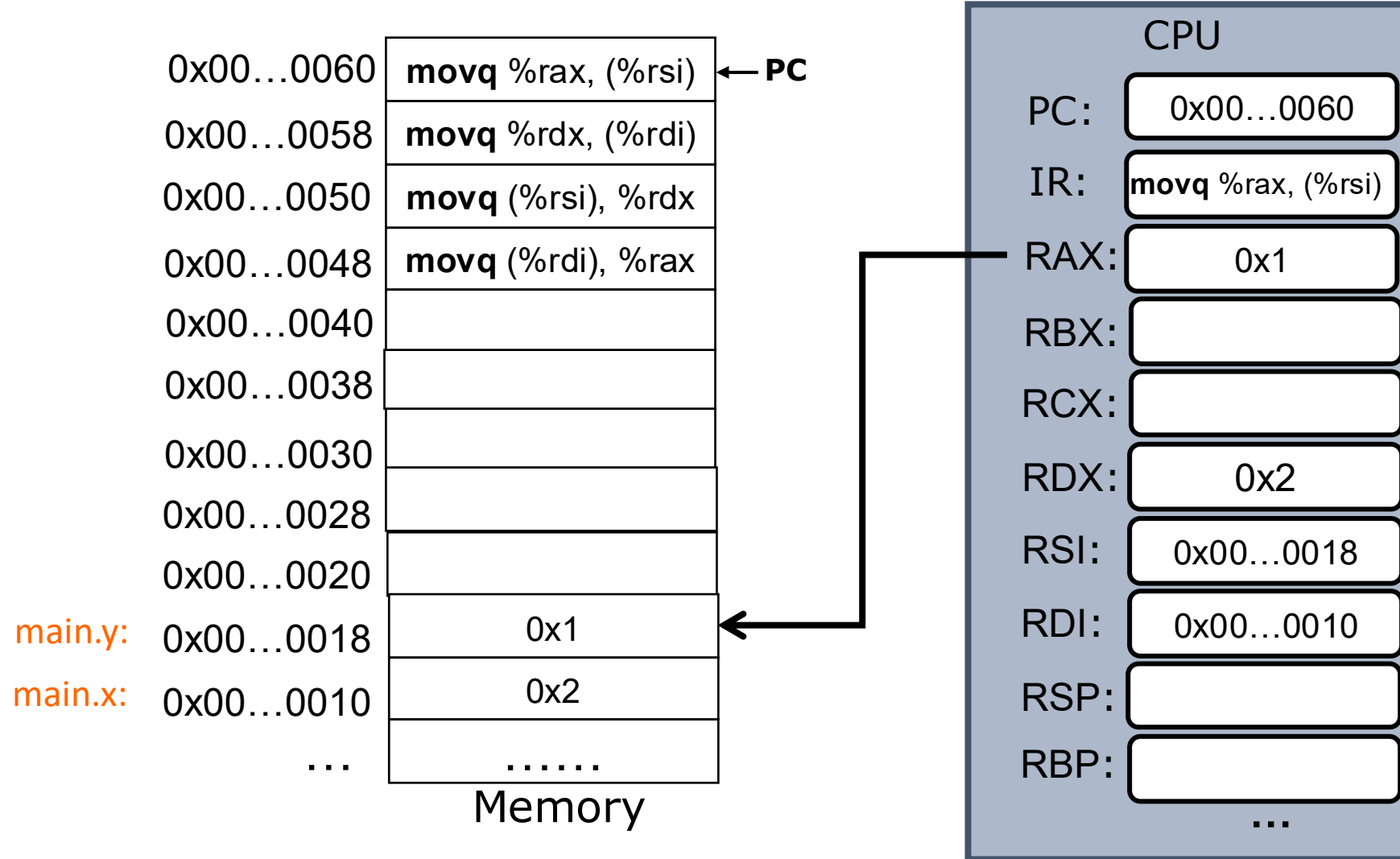
swap func

	0x00...0060	movq %rax, (%rsi) ← PC
	0x00...0058	movq %rdx, (%rdi)
	0x00...0050	movq (%rsi), %rdx
	0x00...0048	movq (%rdi), %rax
	0x00...0040	
	0x00...0038	
	0x00...0030	
	0x00...0028	
	0x00...0020	
main.y:	0x00...0018	0x2
main.x:	0x00...0010	0x2
	...	

Memory

CPU	
PC:	0x00...0060
IR:	movq %rax, (%rsi)
RAX:	0x1
RBX:	
RCX:	
RDX:	0x2
RSI:	0x00...0018
RDI:	0x00...0010
RSP:	
RBP:	
...	

swap func



mov: limitation of direct addressing

Direct addressing

- The address must be calculated and stored in the register before each memory access.

```
long a[3] = {1, 2, 3};  
a[2] = 0;
```



1. Calculate address of a[2]
2. Store it in some register, e.g. %rax
3. Do memory write, e.g. movq \$0, (%rax)

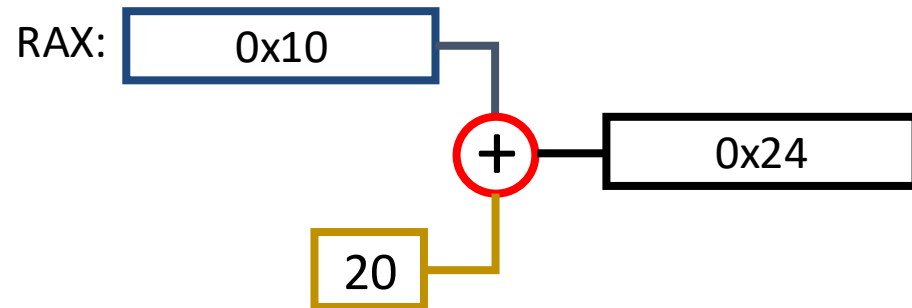
$\&a[2] = \text{address of } a[0] + \text{stride} * 2$

Address mode with displacement

$D(\text{Register}): \text{val}(\text{Register}) + D$

- Register specifies the start of the memory region
- Constant D specifies the offset

Memory Operand: 20(RAX)



Address mode with displacement

$D(\text{Register}): \text{val}(\text{Register}) + D$

- Register specifies the start of the memory region
- Constant D specifies the offset

```
long a[3] = {1, 2, 3};  
a[2] = 0;
```



// Suppose register %rax contains address of a[0]

```
mov $0, 16(%rax)
```

```
for (i = 0; i < 3; i++)  
    a[i] = 1;
```



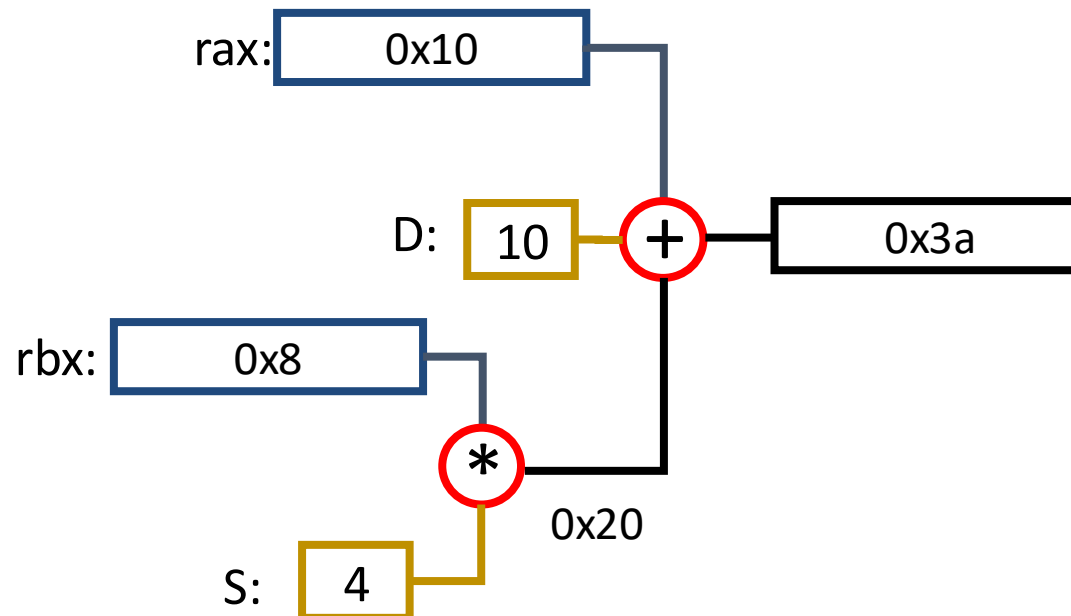
?

X86's Complete Memory Addressing Mode

$$D(Rb, Ri, S): \text{val}(Rb) + S * \text{val}(Ri) + D$$

- Rb: Base register
- D: Constant “displacement”
- Ri: Index register (not **%rsp**)
- S: Scale: 1, 2, 4, or 8

Memory operand: 10(%rax, %rbx, 4)



Complete Memory Addressing Mode

$$D(Rb, Ri, S): \text{val}(Rb) + S * \text{val}(Ri) + D$$

If S is 1 or D is 0, they can be omitted

- (Rb, Ri): $\text{val}(Rb) + \text{val}(Ri)$
- D(Rb, Ri): $\text{val}(Rb) + \text{val}(Ri) + D$
- (Rb, Ri, S): $\text{val}(Rb) + S * \text{val}(Ri)$

Address Computation Examples

%rdx	0xf000
%rcx	0x100

Expression	Address Computation	Address
0x8 (%rdx)		
(%rdx, %rcx)		
(%rdx, %rcx, 4)		
0x80 (, %rdx, 2)		

Address Computation Examples

%rdx	0xf000
%rcx	0x100

Expression	Address Computation	Address
0x8 (%rdx)	0xf000 + 0x8	0xf008
(%rdx, %rcx)		
(%rdx, %rcx, 4)		
0x80 (, %rdx, 2)		

Address Computation Examples

<code>%rdx</code>	<code>0xf000</code>
<code>%rcx</code>	<code>0x100</code>

Expression	Address Computation	Address
<code>0x8(%rdx)</code>	<code>0xf000 + 0x8</code>	<code>0xf008</code>
<code>(%rdx,%rcx)</code>	<code>0xf000 + 0x100</code>	<code>0xf100</code>
<code>(%rdx,%rcx,4)</code>		
<code>0x80(,%rdx,2)</code>		

Address Computation Examples

%rdx	0xf000
%rcx	0x100

Expression	Address Computation	Address
0x8 (%rdx)	0xf000 + 0x8	0xf008
(%rdx, %rcx)	0xf000 + 0x100	0xf100
(%rdx, %rcx, 4)	0xf000 + 4*0x100	0xf400
0x80 (, %rdx, 2)		

Address Computation Examples

<code>%rdx</code>	<code>0xf000</code>
<code>%rcx</code>	<code>0x100</code>

Expression	Address Computation	Address
<code>0x8(%rdx)</code>	<code>0xf000 + 0x8</code>	<code>0xf008</code>
<code>(%rdx,%rcx)</code>	<code>0xf000 + 0x100</code>	<code>0xf100</code>
<code>(%rdx,%rcx,4)</code>	<code>0xf000 + 4*0x100</code>	<code>0xf400</code>
<code>0x80(,%rdx,2)</code>	<code>2*0xf000 + 0x80</code>	<code>0x1e080</code>

Complete addressing mode

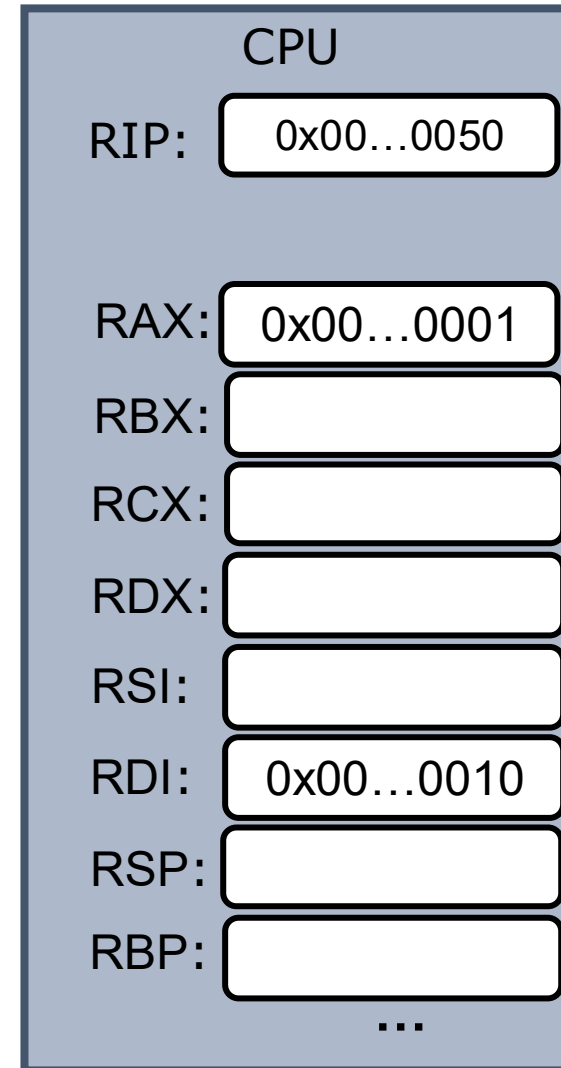
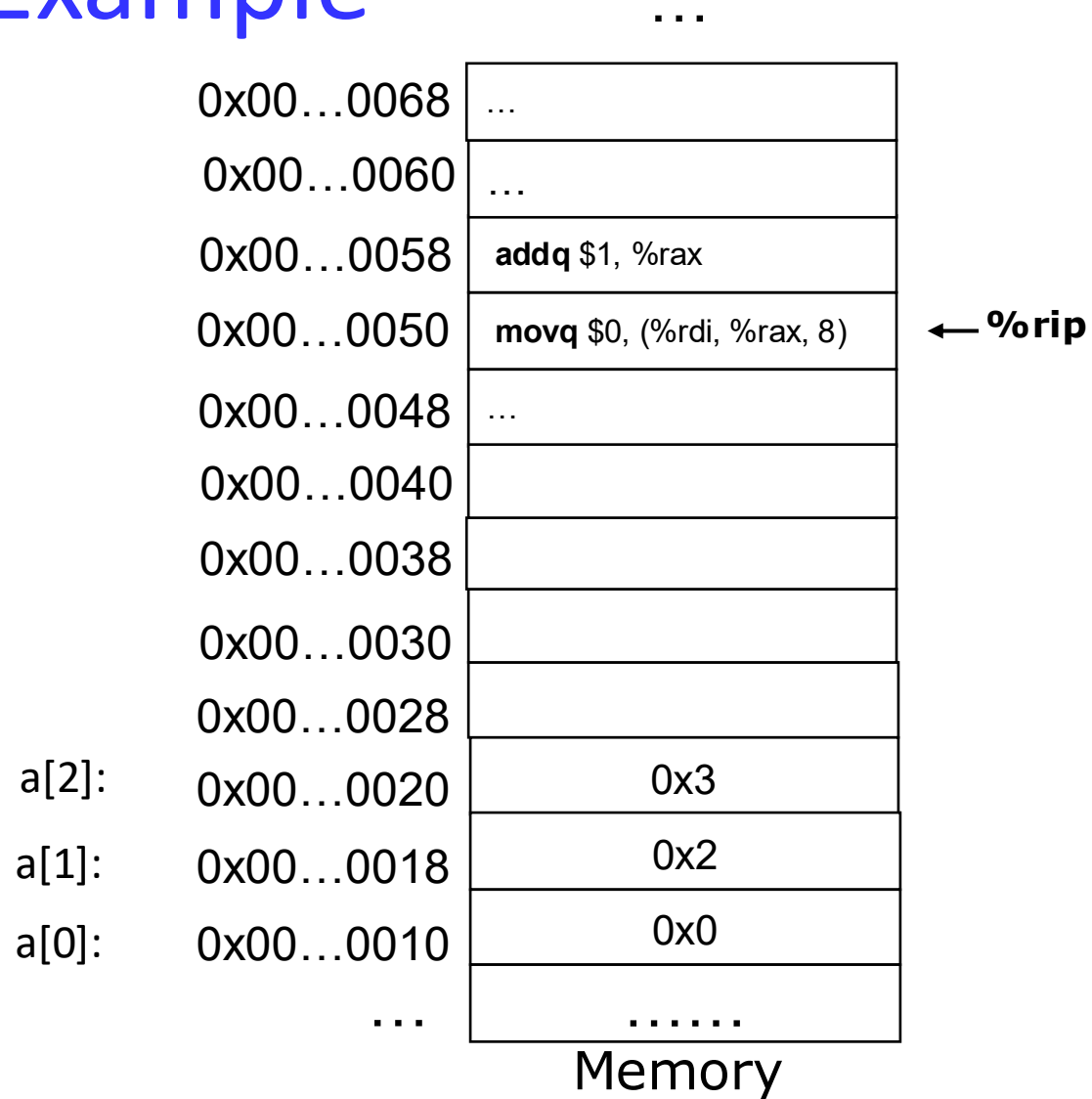
$$D(Rb, Ri, S): \text{val}(Rb) + S * \text{val}(Ri) + D$$

```
long a[] = {1, 2, 3};  
for(int i = 0; i < 3; i++) {  
    a[i] = 0;  
}
```

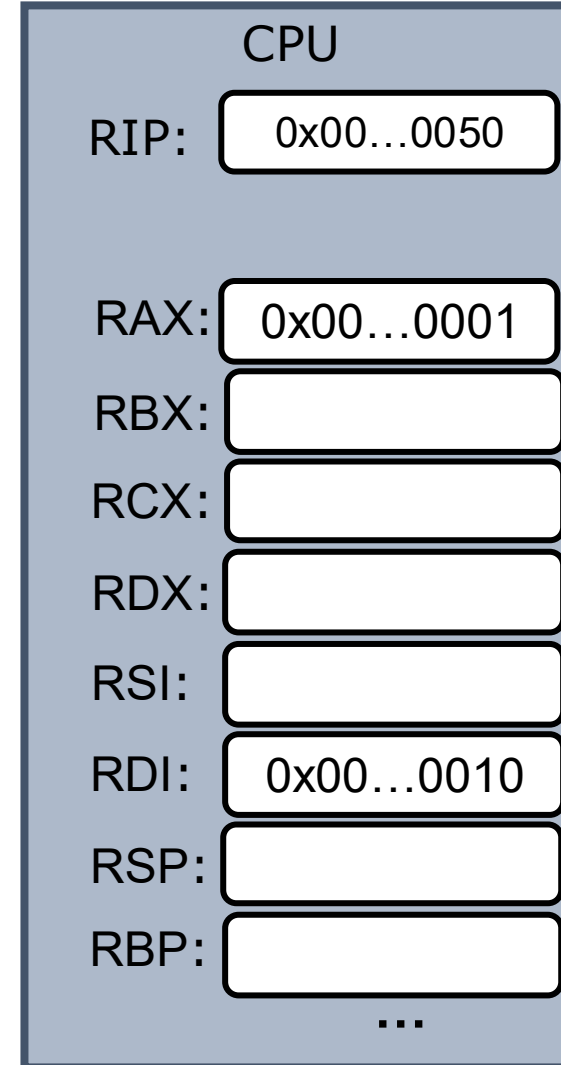
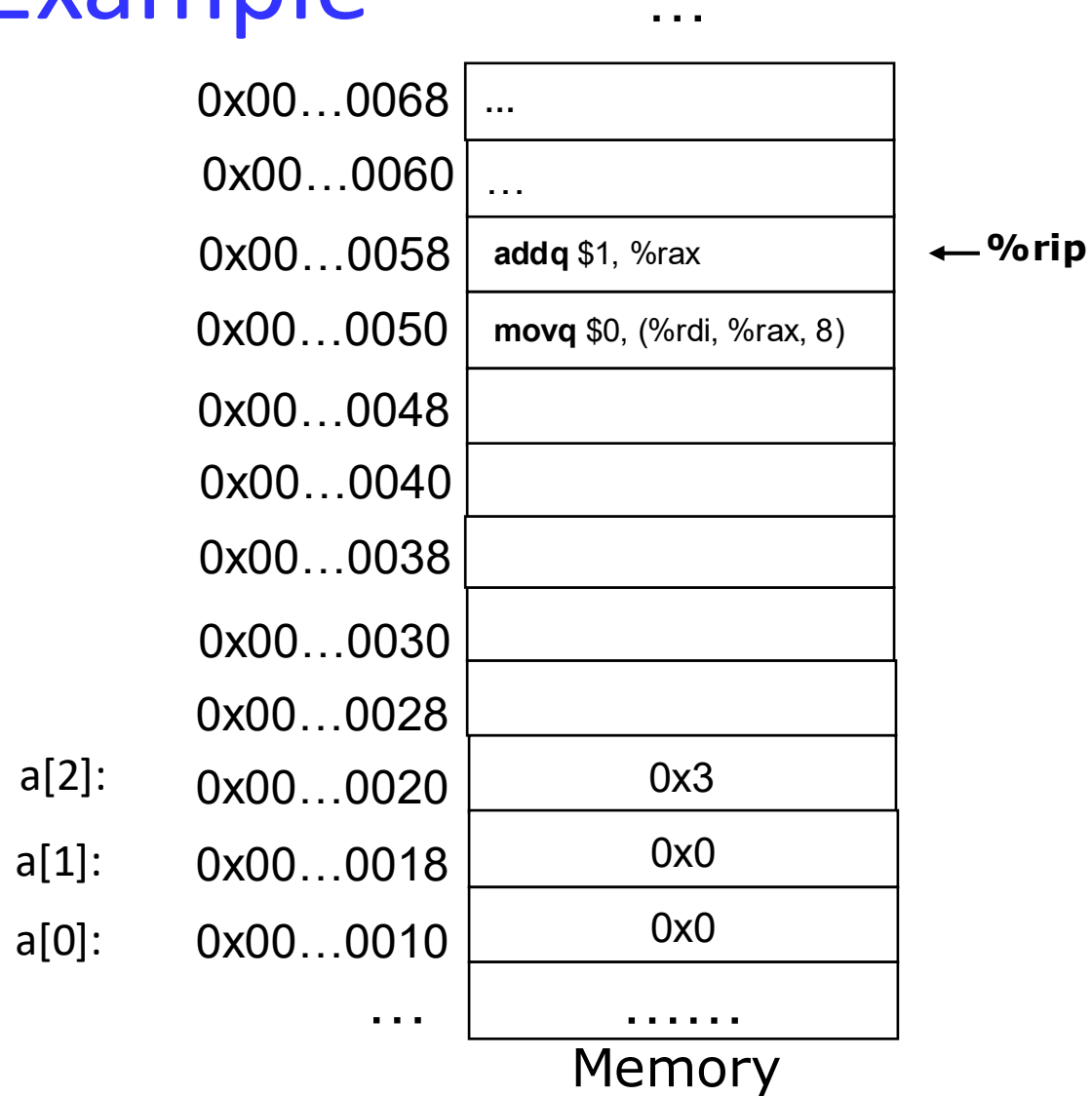


1. Store address of a[0] in some register, say %rdi, store i in another register, say %rax
2. Do memory write: `mov $0, (%rdi, %rax, ?)`

Example



Example

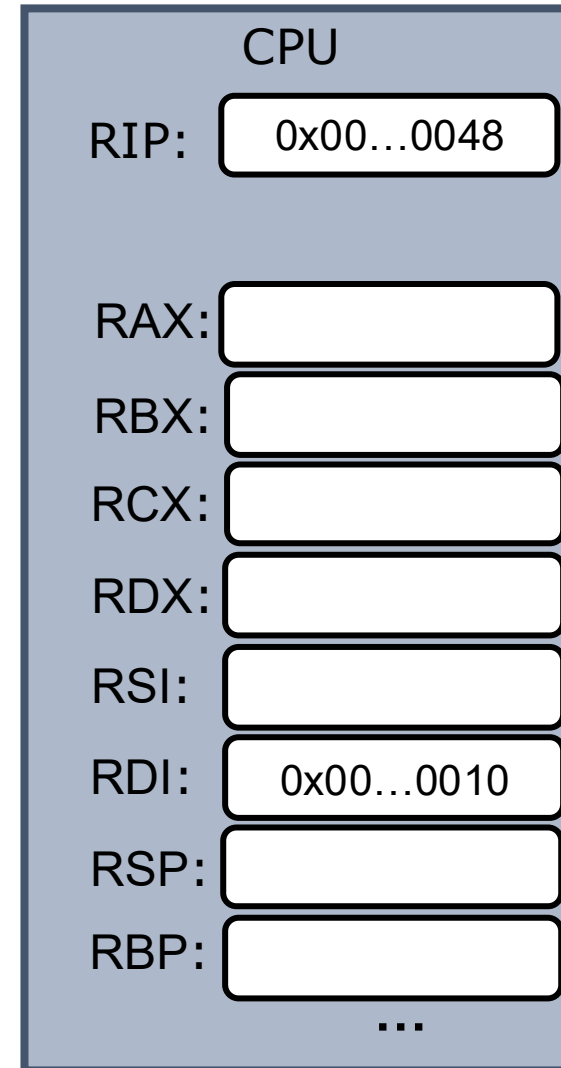
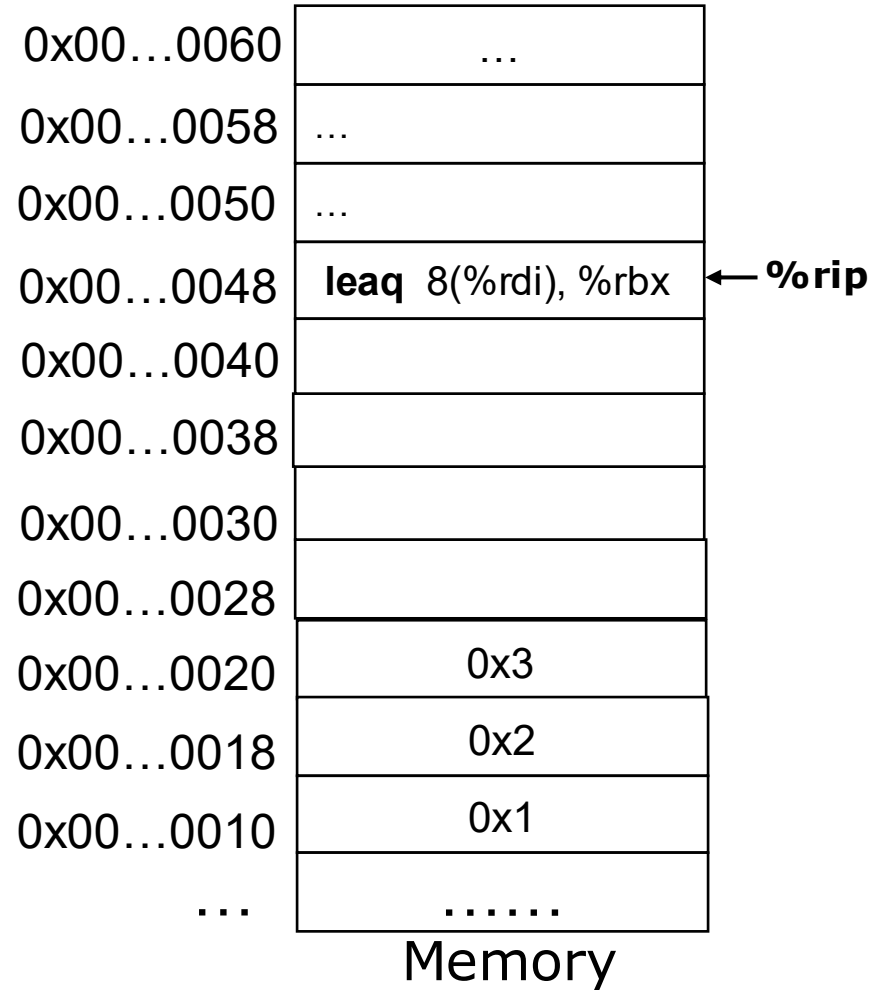


lea instruction

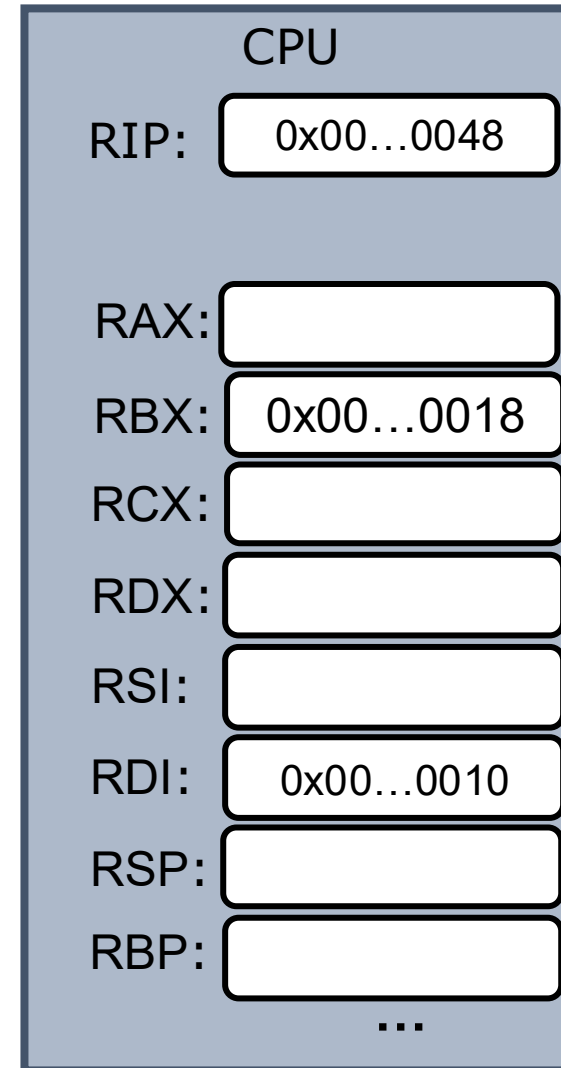
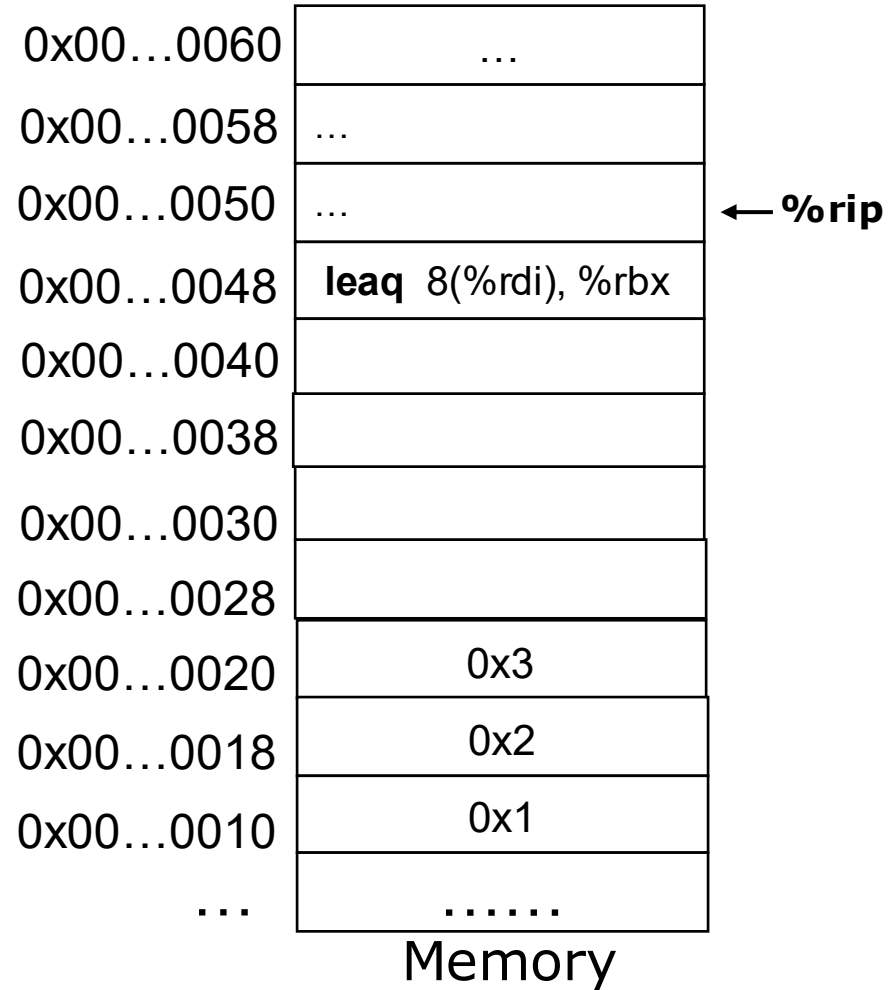
leaq *Source, Dest*

- Short for **L**oad **E**ffective **A**ddress
- Set *Dest* to the address denoted by *Source* address mode expression
- Performs address calculation only; no memory access!

Example



Example



Common use case for lea: calculate address

```
long *p = &a[i];
```



```
leaq (%rdi, %rsi, 8), %rax
```

Another common use case for lea

Lea is used to compute certain simple arithmetic expression

```
long m3(long x)
{
    return x*3;
}
```



```
leaq (%rdi, %rdi, 2), %rax
```

Assume %rdi has the value of x

Arithmetic Expression Puzzle

Suppose %rdi, %rsi, %rax corresponds to variables x, y, s, respectively

```
leaq    (%rdi,%rsi,2), %rax
leaq    (%rax,%rax,4), %rax
```



```
long f(long x, long y)
{
    long s = ??;
    return s;
}
```

Arithmetic Expression Puzzle

Suppose %rdi, %rsi, %rax contains variable x, y, s respectively

```
leaq    (%rdi,%rsi,2), %rax  
leaq    (%rax,%rax,4), %rax
```



```
long f(long x, long y)  
{  
    long s = 5(x + 2y);  
    return s;  
}
```

Basic Arithmetic Operations

addq	Src, Dest	$\text{Dest} = \text{Dest} + \text{Src}$
subq	Src, Dest	$\text{Dest} = \text{Dest} - \text{Src}$
imulq	Src, Dest	$\text{Dest} = \text{Dest} * \text{Src}$
incq	Dest	$\text{Dest} = \text{Dest} + 1$
decq	Dest	$\text{Dest} = \text{Dest} - 1$
negq	Dest	$\text{Dest} = -\text{Dest}$

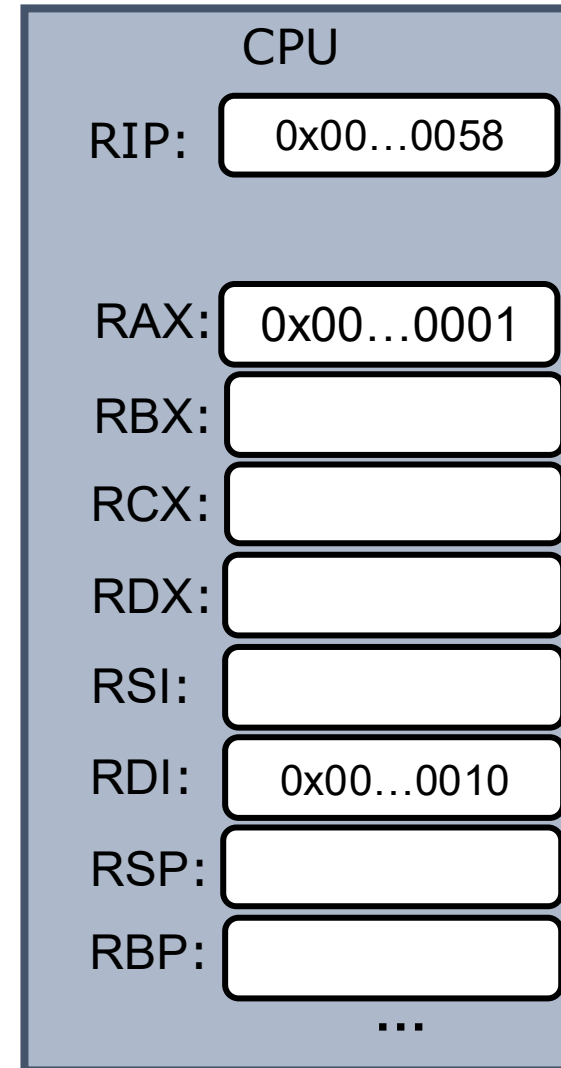
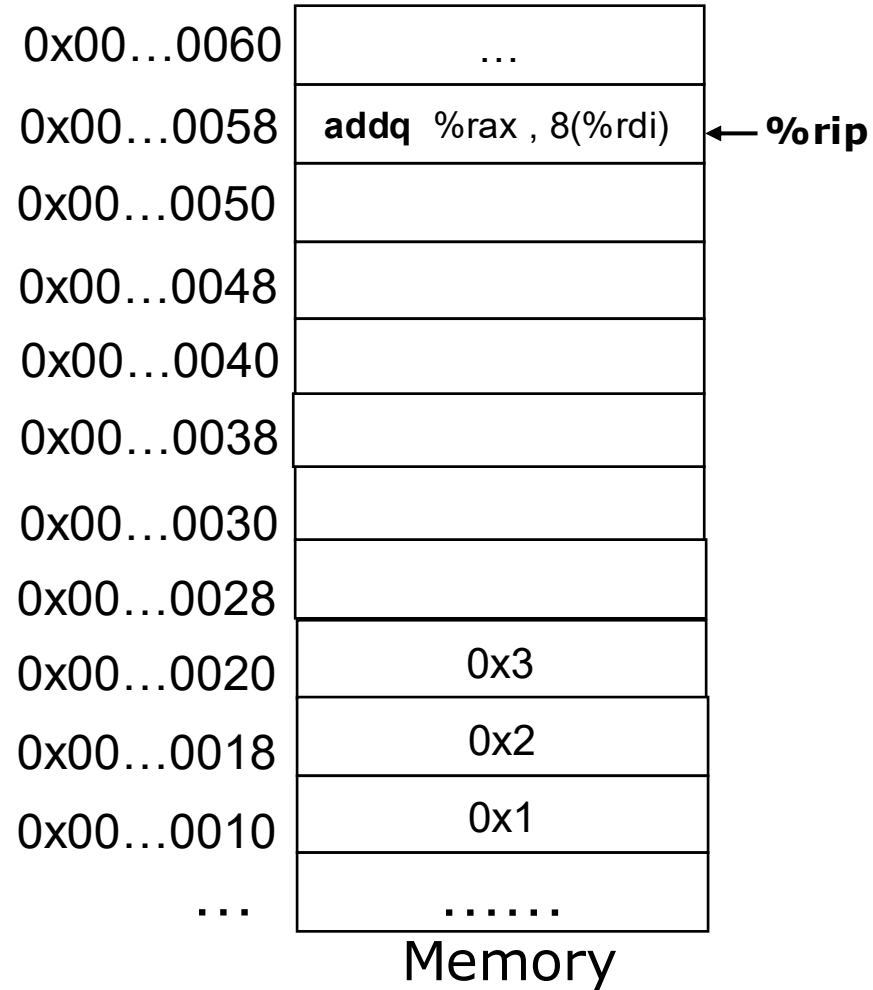
Bitwise Operations

salq	Src, Dest	Dest = Dest << Src
sarq	Src, Dest	Dest = Dest >> Src
shlq	Src, Dest	Dest = Dest << Src
shrq	Src, Dest	Dest = Dest >> Src
xorq	Src, Dest	Dest = Dest ^ Src
andq	Src, Dest	Dest = Dest & Src
orq	Src, Dest	Dest = Dest Src
notq	Dest	Dest = ~Dest

Arithmetic left shift
Arithmetic right shift
Logical left shift
Logical right shift



Example



Example

